

tf.compat.v1.space_to_batch Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/space_to_batch

SpaceToBatch for 4-D tensors of type T.

Function Signatures

```
tf.compat.v1.space_to_batch( input, paddings,  
                             block_size=None, name=None, block_shape=None )
```

Code Examples

```
tf.compat.v1.space_to_batch(  
input, paddings, block_size=None, name=None, block_shape=None  
)
```

```
tf.compat.v1.space_to_batch(  
input, paddings, block_size=None, name=None, block_shape=None  
)
```

```
[batch*block_size*block_size, height_pad/block_size,  
width_pad/block_size,  
depth]
```

```
[batch*block_size*block_size, height_pad/block_size,  
width_pad/block_size,  
depth]
```

```
x = [[[[1], [2]], [[3], [4]]]]
```

```
x = [[[[1], [2]], [[3], [4]]]]
```

```
[[[[1]]], [[2]], [[3]], [[4]]]
```

```
[[[[1]]], [[2]], [[3]], [[4]]]
```

Documentation

SpaceToBatch for 4-D tensors of type T.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.nn.space_to_batch`

This is a legacy version of the more general `SpaceToBatchND`.

Zero-pads and then rearranges (permutes) blocks of spatial data into batch. More specifically, this op outputs a copy of the input tensor where values from the height and width dimensions are moved to the batch dimension. After the zero-padding, both height and width of the input must be divisible by the block size.

The attr `block_size` must be greater than one. It indicates the block size.

- Non-overlapping blocks of size `block_size` x `block_size` in the height and

width dimensions are rearranged into the batch dimension at each location.

- The batch of the output tensor is `batch * block_size * block_size`.
- Both `height_pad` and `width_pad` must be divisible by `block_size`.

The shape of the output will be:

Some examples:

(1) For the following input of shape `[1, 2, 2, 1]` and `block_size` of 2:

The output tensor has shape [4, 1, 1, 1] and value:

(2) For the following input of shape [1, 2, 2, 3] and block_size of 2:

The output tensor has shape [4, 1, 1, 3] and value:

(3) For the following input of shape [1, 4, 4, 1] and block_size of 2:

The output tensor has shape [4, 2, 2, 1] and value:

(4) For the following input of shape [2, 2, 4, 1] and block_size of 2:

The output tensor has shape [8, 1, 2, 1] and value:

Among others, this operation is useful for reducing atrous convolution into regular convolution.

The effective spatial dimensions of the zero-padded input tensor will be:

Returns